

MySQL Sample Database: classicmodels

Introductory Tutorial on the MySQL Sample Database

DSAI1002 – Database Practice

Faculty of Data Science & Artificial Intelligence

2026

Content Source: MySQL Tutorial – MySQL Sample Database.

Learning Objectives

After this lesson, learners will be able to:

- 1 Understand what `classicmodels` is and why it is commonly used for learning MySQL.
- 2 Identify the primary data groups in the database.
- 3 Understand the roles of key tables within the schema.
- 4 Read the ER Diagram to trace relationships between tables.
- 5 Set up the sample database to practice subsequent MySQL lessons.

What is the classicmodels Database?

Core Concept

classicmodels is a sample database modeling a retail business that sells scale model classic vehicles.

This database contains business operations data resembling real-world enterprise systems, making it ideal for practicing SQL queries from foundational to advanced levels.

Data Groups in classicmodels

- Customers
- Products
- Product lines
- Sales orders
- Order details (line items)
- Customer payments
- Employees
- Sales offices
- Reporting structure

Why Use classicmodels?

- 1 More realistic and comprehensive than toy examples with only one or two tables.
- 2 Well-suited for practicing SELECT, WHERE, ORDER BY, GROUP BY, JOIN, subqueries, views, and stored procedures.
- 3 Features realistic primary key–foreign key relationships across multiple tables.
- 4 Easily maps to real business analysis questions: top-spending customers, best-selling products, revenue by country, and unpaid orders.

Downloading the Sample Database

- Download the sample package from the MySQL Tutorial website.
- The package is typically compressed:

`sampledatabase.zip`

- Once downloaded, extract the archive to obtain the SQL script used for importing into MySQL Server.

Extracting the Database File

- You can use an extraction utility such as 7-Zip.
- After extraction, you will typically find:

```
mysqlsampledatabase.sql
```

What does the SQL file contain?

Commands to create the database, create tables, and populate them with sample data.

Two Ways to Import into MySQL Server

Method 1

Using the `mysql` command-line client

- Fast and lightweight
- Ideal for users comfortable with the terminal

Method 2

Using MySQL Workbench

- Graphical and intuitive
- Ideal for beginners

Method 1: Import via Command Line

Assuming the SQL file is located in the Downloads folder:

```
mysql -u root -p < "C:\Users\YourName\Downloads\mysqlsampledatabase.sql"
```

- -u root: Log in as user root.
- -p: Prompt for password.
- <: Redirect SQL file contents into MySQL for execution.

Method 2: Import via MySQL Workbench

- ➊ Open MySQL Workbench and connect to your MySQL Server.
- ➋ Navigate to the menu: `Server > Data Import`.
- ➌ Choose `Import from Self-Contained File` and select `mysqlsampledatabase.sql`.
- ➍ Click `Start Import`.
- ➎ Verify that schema `classicmodels` appears in the schemas panel.

Verification After Import

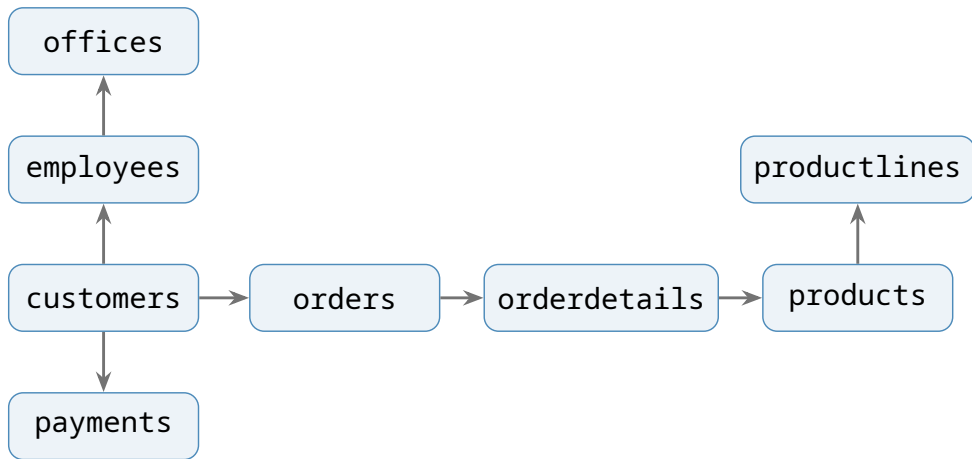
```
SHOW DATABASES;  
USE classicmodels;  
SHOW TABLES;
```

Upon successful import, the table list will include customers, employees, offices, orders, orderdetails, payments, productlines, and products.

Core Tables in the Schema

Table	Purpose
customers	Stores customer account and profile data
products	Stores inventory list of scale model classic vehicles
productlines	Stores product categories and descriptions
orders	Stores sales order headers
orderdetails	Stores line item details for each order
payments	Stores customer payment transactions
employees	Stores employee directory and reporting hierarchy
offices	Stores sales office locations

ER Diagram: High-Level Relationships



Simplified

diagram; the employees table also includes a self-referencing relationship reportsTo.

Customer and Product Tables

customers

Customer ID, customer name, contact details, address, country, sales representative, credit limit.

products

Product code, product name, product line, scale, vendor, quantity in stock, buy price, MSRP.

productlines

Product categories: Classic Cars, Motorcycles, Planes, Ships, Trains, Vintage Cars, etc.

Orders and Payments Tables

orders

Order number, order date, required date, shipped date, status, comments, customer number.

orderdetails

Order number, product code, quantity ordered, price each, order line number.

payments

Customer number, check/payment number, payment date, and paid amount.

Employees and Offices Tables

employees

Employee number, full name, email, job title, assigned office, and direct manager.

offices

Office code, city, phone, address, state/province, country, and territory.

Important Note

The employees table contains a self-referencing foreign key to represent the managerial hierarchy.

Quick Inspection Queries

```
USE classicmodels;
```

```
SHOW TABLES;
```

```
SELECT customerNumber, customerName, country  
FROM customers  
LIMIT 10;
```

Inspecting Products and Recent Orders

```
SELECT productCode, productName, productLine, buyPrice, MSRP  
FROM products  
LIMIT 10;
```

```
SELECT orderNumber, orderDate, status, customerNumber  
FROM orders  
ORDER BY orderDate DESC  
LIMIT 10;
```

Computing Line Item Revenue

```
SELECT orderNumber,  
       productCode,  
       quantityOrdered,  
       priceEach,  
       quantityOrdered * priceEach AS lineRevenue  
FROM orderdetails  
LIMIT 10;
```

Significance

Each record in `orderdetails` represents a product within an order. Line item revenue is calculated as quantity ordered multiplied by unit price.

JOIN: Orders with Customer Names

```
SELECT o.orderNumber,  
        o.orderDate,  
        o.status,  
        c.customerName  
FROM orders o  
JOIN customers c  
  ON o.customerNumber = c.customerNumber  
LIMIT 10;
```

The foreign key `customerNumber` links orders to customers.

JOIN: Products with Product Lines

```
SELECT p.productCode,  
        p.productName,  
        p.productLine,  
        pl.textDescription  
FROM products p  
JOIN productlines pl  
    ON p.productLine = pl.productLine  
LIMIT 10;
```

This query maps each product to its category description in `productlines`.

GROUP BY: Revenue by Order

```
SELECT od.orderNumber,  
       SUM(od.quantityOrdered * od.priceEach) AS totalRevenue  
FROM   orderdetails od  
GROUP BY od.orderNumber  
ORDER BY totalRevenue DESC  
LIMIT 10;
```

Mental Model

Select grouping key → calculate aggregate metric → sort output.

MySQL Topics Explored with classicmodels

Topic	Practical Example
Basic SELECT	Retrieve customer and product catalogs
WHERE	Filter customers by country or credit limit
ORDER BY	Sort orders chronologically
GROUP BY	Calculate revenue per customer or order
JOIN	Connect customers with their orders
Subqueries	Identify customers with above-average purchase value
Views / Procedures	Build reporting summaries and reusable routines
Indexing	Optimize query performance on keys and filters

Exercise 1: Import Database

Task

Import the `classicmodels` database into your MySQL Server instance.

Verify the setup using the following commands:

- `SHOW DATABASES;`
- `USE classicmodels;`
- `SHOW TABLES;`

Exercise 2: Query the First 10 Customers

Retrieve customerNumber, customerName, city, and country.

```
SELECT customerNumber, customerName, city, country  
FROM customers  
LIMIT 10;
```

Exercises 3–4: Products and Revenue

Exercise 3

Query products belonging to the Classic Cars product line. Display productCode, productName, productLine, and MSRP.

Exercise 4

Calculate the total revenue for each order from orderdetails using quantityOrdered * priceEach, grouped by orderNumber.

Exercise 5: JOIN Orders and Customers

Write a JOIN query to display orders along with corresponding customer names.

- Table 1: orders
- Table 2: customers
- Join key: customerNumber

Discussion Questions

- 1 What business model does the `classicmodels` database simulate?
- 2 Why is `classicmodels` well-suited for learning relational databases?
- 3 What role does the `orderdetails` table serve?
- 4 Why is the `productlines` table separated from the `products` table?
- 5 How does an ER Diagram help when working with `classicmodels`?

Multiple Choice Questions

- 1 Which industry domain does classicmodels model?
A. Scale model classic car retail B. Hospital management C. Digital library D. Social network
- 2 Which table stores customer data?
A. products B. customers C. orders D. offices
- 3 Which table stores line items within each order?
A. payments B. employees C. orderdetails D. productlines
- 4 Which table records customer payment transactions?
A. products B. orders C. offices D. payments

Lesson Summary

- `classicmodels` is a widely used sample database in MySQL simulating a scale model vehicle retailer.
- The schema comprises `customers`, `products`, `productlines`, `orders`, `orderdetails`, `payments`, `employees`, and `offices`.
- It provides an ideal dataset for mastering `SELECT`, `JOIN`, `GROUP BY`, aggregations, views, and stored procedures.
- ER Diagrams help learners map entity relationships before composing SQL statements.

Key Terms

MySQL classicmodels Schema

ER Diagram SELECT JOIN

GROUP BY View Stored Procedure